

IACMS

Inter-Agency Case Management System

Full Technical Design Document
Version 2.0 — Complete Production Reference

Architecture · Database Design · API Specifications · Security · Operations

Table of Contents

Table of Contents.....	2
1. Introduction.....	5
1.1 Purpose.....	5
1.2 Scope.....	5
2. System Overview.....	5
2.1 Core Concept.....	5
2.2 Platform Stack.....	5
3. System Architecture.....	5
3.1 High-Level Component Diagram.....	5
3.2 Request Flow Sequence.....	6
3.3 Kafka Event Flow.....	7
3.4 Primary Runtime Dependencies.....	7
4. Repository Structure.....	7
5. Service Breakdown.....	8
6. Database Design.....	9
6.1 Entity Relationship Overview.....	9
6.2 Entity Schemas.....	9
Tenant.....	9
User.....	10
Role.....	10
Workflow.....	11
WorkflowVersion.....	11
WorkflowStep.....	12
WorkflowTransition.....	12
Case.....	13
CaseHistory.....	13
OutboxEvent.....	14
AuditEvent.....	14
6.3 Junction Tables.....	15
6.4 Indexing Strategy.....	15
7. Core Data Entities.....	16
8. Multi-Tenancy Architecture.....	16
8.1 Isolation Model Selection.....	16
8.2 Tenant Isolation Rules.....	17
8.3 Tenant Provisioning Flow.....	17
8.4 Cross-Tenant Data Considerations.....	17
9. Workflow Engine.....	17
9.1 Internal Transition Processing.....	17
9.2 Transition Validation Rules.....	18
9.3 Transition Storage and Evaluation.....	18
9.4 Database Transaction Scope.....	18
9.5 Conditional Branching.....	18
9.6 Case Creation.....	19
10. Workflow Versioning Lifecycle.....	19
10.1 Version States.....	19
10.2 State Descriptions.....	19
10.3 Publishing Process.....	19
10.4 Immutability After Publication.....	20

10.5 Case Migration.....	20
11. “Other Side Visibility” Tracking.....	20
11.1 What Users See.....	20
11.2 Example API Response (GET /api/cases/:id).....	20
12. Authentication and Access Control.....	21
12.1 Authentication Modes.....	21
12.2 Token Lifecycle.....	21
12.3 Gateway Validation Flow.....	21
12.4 Tenant Context Injection.....	22
13. Role-Based Access Control Model.....	22
13.1 Permission Model Structure.....	22
13.2 Permission Layers.....	22
13.3 Runtime Evaluation.....	22
13.4 Permission Model Properties.....	23
14. Security Design.....	23
14.1 Password Security.....	23
14.2 JWT Security.....	23
14.3 Transport Security.....	23
14.4 Input Security.....	24
14.5 Session Security.....	24
14.6 Rate Limiting.....	24
14.7 Secret Management.....	24
14.8 Tenant Isolation as a Security Control.....	24
15. API Specifications.....	25
15.1 Auth Service (/api/auth).....	25
15.2 RBAC Service (/api/roles, /api/users).....	25
15.3 Workflow Service (/api/workflows).....	26
15.4 Case Service (/api/cases).....	27
Case Transition Request / Response Example.....	27
15.5 Referral Service (/api/referrals).....	28
15.6 Audit Service (/api/audit).....	28
15.7 Notification Service (/api/notifications).....	28
15.8 Common HTTP Response Codes.....	29
16. Event-Driven Architecture.....	29
16.1 Kafka Topics.....	29
16.2 Producer and Consumer Map.....	30
16.3 Event Types.....	30
16.4 Standard Event Envelope.....	31
16.5 Reliability and Error Handling.....	31
17. Transaction Management.....	32
17.1 The Distributed Transaction Problem.....	32
17.2 The Transactional Outbox Pattern.....	32
17.3 Outbox Worker.....	32
17.4 Saga Pattern for Multi-Service Flows.....	32
18. Failure Handling and Resilience.....	33
18.1 Circuit Breaker Pattern.....	33
18.2 Per-Dependency Failure Scenarios.....	33
18.3 Retry Policy.....	34
18.4 Health Check Endpoints.....	34
18.5 Graceful Shutdown.....	34
19. Performance Considerations.....	34

19.1 Database.....	34
19.2 Caching Strategy.....	35
19.3 Pagination.....	35
19.4 Expected Workload Characteristics.....	35
20. Observability and Monitoring.....	35
20.1 Structured Logging.....	35
20.2 Correlation IDs.....	36
20.3 Metrics (Prometheus).....	36
20.4 Distributed Tracing.....	36
20.5 Alerting Thresholds.....	37
21. Deployment Architecture.....	37
21.1 Kubernetes Architecture.....	37
21.2 Kubernetes Resource Summary.....	38
21.3 Environment Strategy.....	38
21.4 CI/CD Pipeline.....	38
22. Scalability Strategy.....	39
22.1 Horizontal Service Scaling.....	39
22.2 Database Scaling.....	39
22.3 Kafka Scaling.....	39
22.4 Redis Scaling.....	39
22.5 Long-Term Tenant Growth.....	40
23. Architecture Decision Records.....	40
ADR-001: Microservices Architecture.....	40
ADR-002: Node.js as Application Runtime.....	40
ADR-003: PostgreSQL as Primary Database.....	41
ADR-004: Apache Kafka as Event Bus.....	41
ADR-005: Redis for Session and Cache.....	42
ADR-006: Prisma as ORM.....	42
24. Audit and Logging.....	43
24.1 Why Audit Logging Matters.....	43
24.2 What Is Logged.....	43
24.3 Audit Architecture.....	43
25. Notifications.....	43
25.1 Notification Types.....	43
25.2 Trigger Events.....	43
26. Local Development Setup.....	44
26.1 Prerequisites.....	44
26.2 Start Infrastructure Services.....	44
26.3 Apply Database Migrations and Seed Data.....	44
26.4 Start Individual Services.....	44
26.5 Useful Root Commands.....	44
27. API Conventions.....	44
28. Testing and Validation.....	45
29. Documentation Map.....	45
30. Current Project Status.....	45
31. Risks and Challenges.....	46
32. Notes for Future Development.....	46
33. Conclusion.....	46

1. Introduction

1.1 Purpose

This document is the complete technical reference for IACMS (Inter-Agency Case Management System). It covers system architecture, database design, API specifications, workflow engine internals, security model, event-driven integration, deployment strategy, and operational guidance. It is intended for software engineers, system architects, and DevOps engineers working on or integrating with the platform.

1.2 Scope

The system is designed to:

- Support multiple tenants (organizations) on a single shared platform with strict data isolation.
- Allow each tenant to define custom workflows independently.
- Manage cases that progress through tenant-defined workflow steps.
- Track complete case history with full auditability.
- Provide cross-agency collaboration through referral flows.
- Expose well-defined REST APIs and event streams for external integration.

2. System Overview

2.1 Core Concept

IACMS is organized around three functional layers: a Case Management Layer that handles case creation, storage, and tracking; a Workflow Definition Layer that allows tenants to configure custom steps and transition rules; and a Workflow Execution Layer that enforces those rules, moves cases between steps, and logs every action.

2.2 Platform Stack

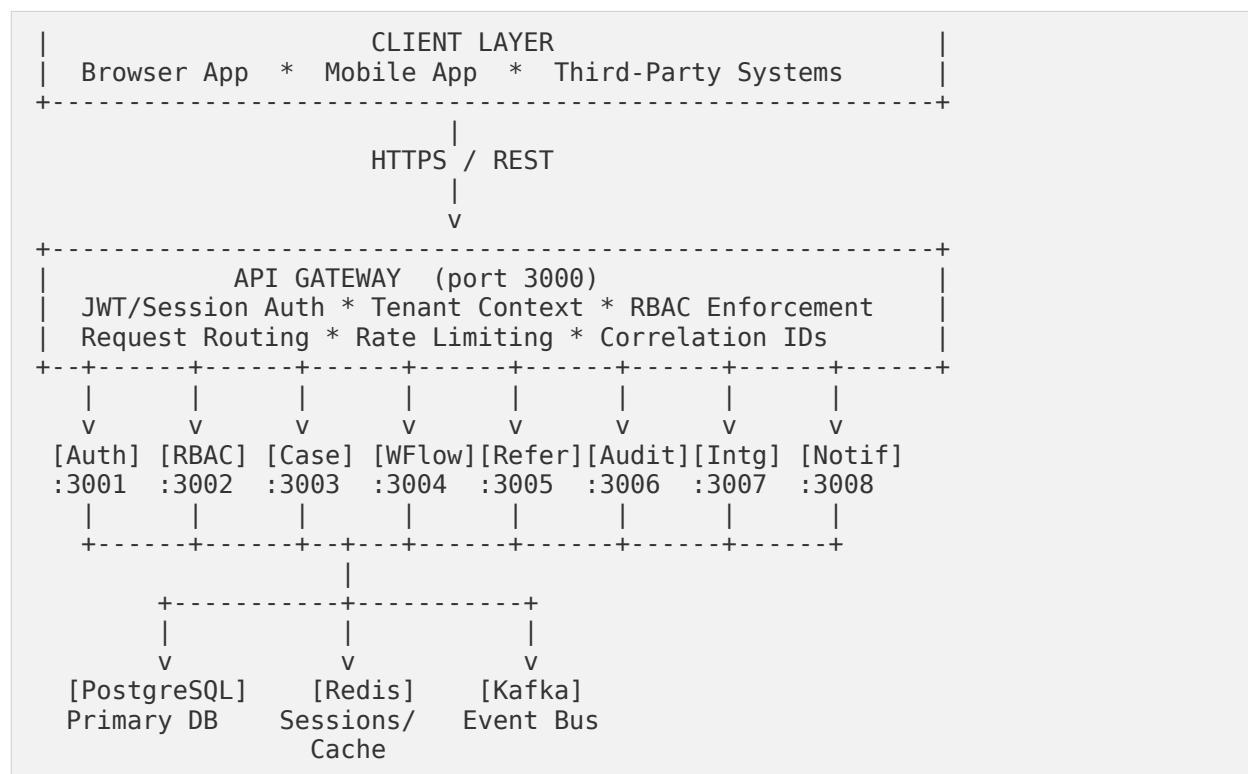
The platform is implemented as a Node.js microservices-based backend, combining a central API Gateway with nine domain-specific services backed by PostgreSQL, Redis, and Kafka.

3. System Architecture

3.1 High-Level Component Diagram

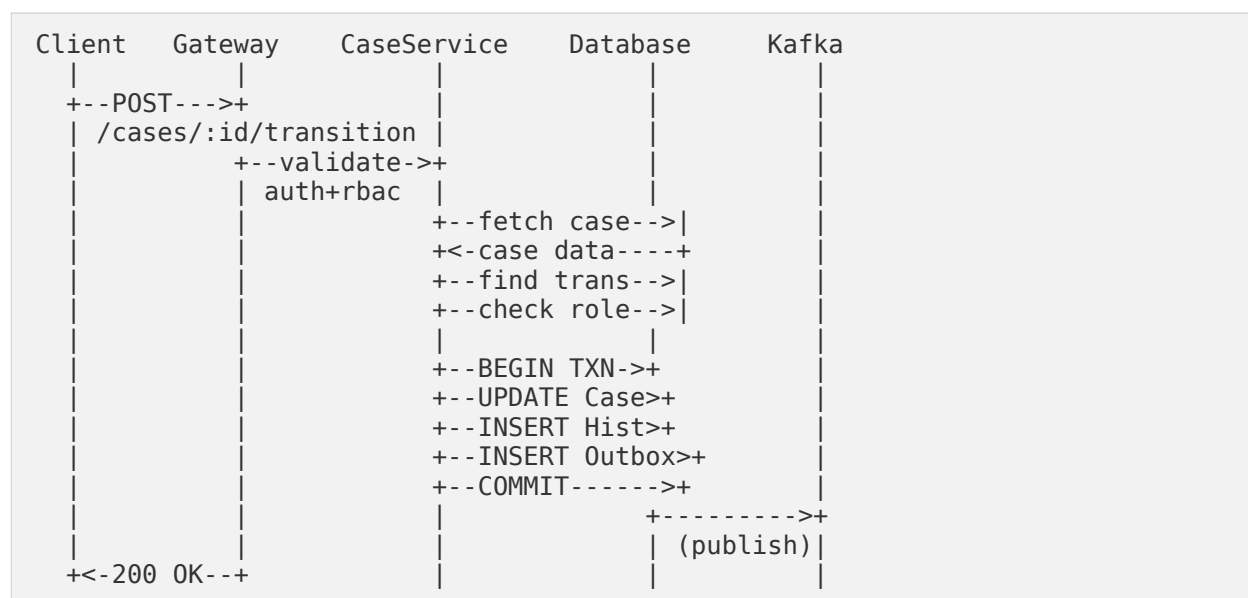
The diagram below shows the relationships between the client layer, the API Gateway, the backend microservices, and the shared infrastructure components.

+-----+



3.2 Request Flow Sequence

The sequence below illustrates the complete internal processing path for a case transition request, from client to persistence and asynchronous event dispatch.



3.3 Kafka Event Flow

The diagram below maps each microservice to the Kafka topics it produces to and the topics it consumes from.

Producers	Topics	Consumers
-----	-----	-----
Case Svc	--> iacms.case.events	--> Audit Svc --> Notif Svc
WFlow Svc	--> iacms.workflow.events	--> Audit Svc
Ref Svc	--> iacms.referral.events	--> Audit Svc --> Notif Svc --> Case Svc
All Svcs	--> iacms.audit.ingestion	--> Audit Svc
Case/Ref	--> iacms.notification.dispatch	--> Notif Svc

3.4 Primary Runtime Dependencies

Component	Role in the System
Node.js 20+ / Express	Application runtime and HTTP server for each service
PostgreSQL 15+	Primary transactional data store for all persistent state
Prisma ORM	Type-safe database access, schema management, and migrations
Redis 7+	Session storage, distributed caching, and short-lived state
Apache Kafka	Durable asynchronous event bus for inter-service communication
Docker / Kubernetes	Containerisation and orchestration for all environments

4. Repository Structure

The codebase is a modular monorepo separating independently deployable services from shared cross-cutting code:

IACMS/	
├── services/	# Microservices
│ ├── api-gateway/	
│ ├── auth-service/	
│ ├── rbac-service/	
│ ├── case-service/	
│ └── workflow-service/	

```

├── referral-service/
├── audit-service/
├── integration-service/
├── notification-service/
├── shared/                # Cross-service shared code
│   ├── common/
│   ├── middleware/
│   ├── contracts/        # Kafka event schemas
│   └── utils/
├── prisma/                # Prisma schema and migrations
├── infrastructure/        # Docker Compose and K8s configs
├── docs/                  # Supplementary documentation
├── scripts/               # Setup and operational scripts
└── package.json           # Root workspace configuration

```

5. Service Breakdown

Each service is independently deployable and owns a distinct bounded context within the platform:

Service	Port	Responsibility
API Gateway	3000	Single public entry point: auth validation, tenant context injection, RBAC enforcement, request routing, rate limiting, correlation ID assignment
Auth Service	3001	User registration and login, JWT issuance, refresh token management, session creation and invalidation
RBAC Service	3002	Role and permission management, user-role assignments, permission evaluation support
Case Service	3003	Case lifecycle management: creation, workflow step transitions, assignment, history tracking
Workflow Service	3004	Workflow definitions, step and transition configuration, versioning, and publication
Referral Service	3005	Cross-tenant and cross-agency case referrals: creation, acceptance, rejection
Audit Service	3006	Ingestion and storage of structured audit events, query interface for compliance reporting
Integration Service	3007	External system integrations, webhook support, and third-party API adapters
Notification Service	3008	Real-time WebSocket notifications and email dispatch triggered by case and referral events

6. Database Design

6.1 Entity Relationship Overview

The following diagram shows the primary relationships between the core data entities:

ENTITY RELATIONSHIP OVERVIEW

```
Tenant --1:N--> User
Tenant --1:N--> Role
Tenant --1:N--> Workflow
Tenant --1:N--> Case

User <--M:N via UserRole--> Role
Role <--M:N via RolePermission--> Permission

Workflow --1:N--> WorkflowVersion
WorkflowVersion --1:N--> WorkflowStep
WorkflowVersion --1:N--> WorkflowTransition
    .fromStepId -----> WorkflowStep
    .toStepId -----> WorkflowStep
    .requiredRoleId -----> Role
WorkflowStep <--M:N via StepRole--> Role

Case --FK--> WorkflowVersion
Case --FK--> WorkflowStep (currentStep)
Case --1:N--> CaseHistory
    .fromStepId -----> WorkflowStep (nullable)
    .toStepId -----> WorkflowStep
    .performedBy -----> User

Case --1:N--> Referral (as source)
Referral --> Case (as target, nullable)
```

6.2 Entity Schemas

Tenant

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
name	VARCHAR(255)	NOT NULL	Display name of the organization
slug	VARCHAR(100)	UNIQUE, NOT NULL	URL-safe identifier (e.g. 'ministry-of-health')
isActive	BOOLEAN	DEFAULT true	Whether the tenant account is currently active

Column	Type	Constraints	Description
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp
updatedAt	TIMESTAMP	NOT NULL	Last modification timestamp

User

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
tenantId	UUID	FK -> Tenant.id, NOT NULL	Owning tenant
email	VARCHAR(255)	NOT NULL	Login email address
passwordHash	VARCHAR(255)	NOT NULL	Bcrypt hash (cost factor 12)
firstName	VARCHAR(100)	NULLABLE	Given name
lastName	VARCHAR(100)	NULLABLE	Family name
isActive	BOOLEAN	DEFAULT true	Account status flag
lastLoginAt	TIMESTAMP	NULLABLE	Timestamp of most recent login
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp
updatedAt	TIMESTAMP	NOT NULL	Last modification timestamp

Constraint: UNIQUE (tenantId, email) — email uniqueness is scoped per tenant.

Role

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
tenantId	UUID	FK -> Tenant.id, NOT NULL	Owning tenant
name	VARCHAR(100)	NOT NULL	Role label (e.g. Officer, Supervisor)
description	TEXT	NULLABLE	Human-readable description of responsibilities
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp

Column	Type	Constraints	Description
updatedAt	TIMESTAMP	NOT NULL	Last modification timestamp

Constraint: UNIQUE (tenantId, name) — role names are unique within a tenant.

Workflow

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
tenantId	UUID	FK -> Tenant.id, NOT NULL	Owning tenant
name	VARCHAR(255)	NOT NULL	Workflow name (e.g. Investigation Process)
description	TEXT	NULLABLE	Purpose and intended use
createdBy	UUID	FK -> User.id	User who created the workflow
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp
updatedAt	TIMESTAMP	NOT NULL	Last modification timestamp

WorkflowVersion

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
workflowId	UUID	FK -> Workflow.id, NOT NULL	Parent workflow
version	INTEGER	NOT NULL	Monotonically increasing version number
status	ENUM	NOT NULL	DRAFT PUBLISHED DEPRECATED
publishedAt	TIMESTAMP	NULLABLE	Timestamp when status changed to PUBLISHED
publishedBy	UUID	FK -> User.id, NULLABLE	User who published this version
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp
updatedAt	TIMESTAMP	NOT NULL	Last modification timestamp

Constraint: UNIQUE (workflowId, version). Only one version may have status = PUBLISHED at a time.

WorkflowStep

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
workflowVersionId	UUID	FK -> WorkflowVersion.id, NOT NULL	Parent workflow version
name	VARCHAR(100)	NOT NULL	Step name (e.g. Review, Approval)
description	TEXT	NULLABLE	What happens at this step
isInitial	BOOLEAN	DEFAULT false	Entry point of the workflow (exactly one per version)
isFinal	BOOLEAN	DEFAULT false	Terminal step; cases here are considered closed
sortOrder	INTEGER	NOT NULL	Display ordering
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp

WorkflowTransition

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
workflowVersionId	UUID	FK -> WorkflowVersion.id, NOT NULL	Parent workflow version
fromStepId	UUID	FK -> WorkflowStep.id, NOT NULL	Source step
toStepId	UUID	FK -> WorkflowStep.id, NOT NULL	Destination step
actionLabel	VARCHAR(100)	NOT NULL	User-facing button label (e.g. Approve, Reject)
requiredRoleId	UUID	FK -> Role.id, NOT NULL	Role a user must hold to trigger this transition
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp

Constraint: UNIQUE (workflowVersionId, fromStepId, actionLabel) — each action label is unique per step per version.

Case

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
tenantId	UUID	FK -> Tenant.id, NOT NULL	Owning tenant
workflowVersionId	UUID	FK -> WorkflowVersion.i d, NOT NULL	Workflow version this case follows
currentStepId	UUID	FK -> WorkflowStep.id, NOT NULL	Current position in the workflow
title	VARCHAR(500)	NOT NULL	Case title
description	TEXT	NULLABLE	Case description
caseData	JSONB	NULLABLE	Tenant-defined structured data fields
status	ENUM	NOT NULL	OPEN IN_PROGRESS CLOSED
assignedUserId	UUID	FK -> User.id, NULLABLE	Currently assigned handler
createdBy	UUID	FK -> User.id, NOT NULL	User who created the case
closedAt	TIMESTAMP	NULLABLE	Timestamp when case reached a final step
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp
updatedAt	TIMESTAMP	NOT NULL	Last modification timestamp

CaseHistory

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
caseId	UUID	FK -> Case.id, NOT NULL	Case this entry belongs to
fromStepId	UUID	FK -> WorkflowStep.id, NULLABLE	Previous step (null on initial case creation)
toStepId	UUID	FK -> WorkflowStep.id, NOT NULL	Step reached after this action

Column	Type	Constraints	Description
transitionId	UUID	FK -> WorkflowTransition.id, NULLABLE	Transition that was triggered
performedBy	UUID	FK -> User.id, NOT NULL	User who performed the action
comment	TEXT	NULLABLE	Optional comment provided by the actor
createdAt	TIMESTAMP	NOT NULL	Timestamp of the action

OutboxEvent

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
eventType	VARCHAR(100)	NOT NULL	Event type string (e.g. CASE_TRANSITIONED)
tenantId	UUID	NOT NULL	Tenant context for partitioning
payload	JSONB	NOT NULL	Full event payload to publish to Kafka
published	BOOLEAN	DEFAULT false	Whether the event has been dispatched
publishedAt	TIMESTAMP	NULLABLE	Timestamp when dispatched to Kafka
createdAt	TIMESTAMP	NOT NULL	Record creation timestamp

AuditEvent

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique identifier
tenantId	UUID	NOT NULL, INDEX	Owning tenant (indexed for filtered queries)
eventType	VARCHAR(100)	NOT NULL	Event classification (e.g. CASE_TRANSITIONED)
entityType	VARCHAR(50)	NOT NULL	Affected entity type (CASE, WORKFLOW, USER...)
entityId	UUID	NOT NULL	ID of the affected entity
performedBy	UUID	NULLABLE	User responsible (null for system-generated events)

Column	Type	Constraints	Description
metadata	JSONB	NULLABLE	Additional context: step names, comments, IP address
createdAt	TIMESTAMP	NOT NULL, INDEX	Event timestamp (indexed for time-range queries)

6.3 Junction Tables

Junction Table	Relationship Encoded
UserRole (userId, roleId)	M:N between User and Role. A user may hold multiple roles; a role may be assigned to multiple users.
RolePermission (roleId, permissionId)	M:N between Role and Permission. Grants fine-grained permissions to roles.
WorkflowStepRole (stepId, roleId)	M:N between WorkflowStep and Role. Defines which roles may view and act on cases at a given step.

6.4 Indexing Strategy

Index	Purpose
UNIQUE (tenantId, email) on User	Enforce per-tenant email uniqueness; fast user lookup at login
(tenantId) on User, Workflow, Case	All primary tenant-scoped list queries filter by tenantId
(tenantId, status) on Case	Dashboard queries filter open/closed cases by tenant
(currentStepId) on Case	Efficiently find all cases at a specific workflow step
(workflowVersionId) on WorkflowStep, WorkflowTransition	Load all steps/transitions for a workflow version in one query
(fromStepId) on WorkflowTransition	Engine lookup: given current step, find all valid transitions
(caseId) on CaseHistory	Load full history for a case in one index scan
(caseId, createdAt) on CaseHistory	Ordered history retrieval; time-range audit queries
(tenantId, createdAt) on AuditEvent	Compliance queries scoped by tenant and time range
(entityType, entityId) on AuditEvent	Look up all audit events for a specific entity

Index	Purpose
(published, createdAt) on OutboxEvent	Outbox worker scans for unpublished events in order

7. Core Data Entities

The following entities form the central business model. Their relationships are fully specified in Section 6.

Entity	Description
Tenant	An organization using the platform. All data is scoped to a tenant.
User	A system user belonging to a tenant.
Role	A named responsibility label (e.g. Officer, Supervisor) assigned to users.
Permission	A discrete capability (e.g. case:create). Grouped into roles.
Workflow	A named workflow definition owned by a tenant.
WorkflowVersion	An immutable snapshot of a workflow once published. Cases reference a specific version.
WorkflowStep	A single stage in a workflow (e.g. Review, Approval).
WorkflowTransition	A permitted movement between two steps, gated by a role.
Case	A business record progressing through a workflow.
CaseHistory	An immutable log entry for every step change a case undergoes.
OutboxEvent	A pending Kafka message, stored in the database for reliable dispatch.
AuditEvent	A compliance log entry for any significant platform action.
Referral	A cross-agency case handoff request between two tenants.

8. Multi-Tenancy Architecture

8.1 Isolation Model Selection

Three multi-tenancy models were evaluated:

Model	Isolation Level	Operational Complexity	When to Use
Shared DB, Row-	Medium — enforced	Low — one schema	Many tenants, moderate

Model	Isolation Level	Operational Complexity	When to Use
Level (chosen)	by application	to manage	compliance needs
Schema-per-Tenant	High — PostgreSQL schema isolation	Medium — migration per tenant	Fewer tenants, stricter isolation needs
Database-per-Tenant	Maximum — full DB isolation	High — separate connections/backups	Regulated industries, large tenants

Row-level isolation was selected because IACMS targets many small-to-medium government agencies, making operational simplicity and cross-tenant platform analytics more valuable than maximum isolation. Tenant filtering is enforced at the application layer on every query, with `tenantId` in every `WHERE` clause. PostgreSQL Row Level Security (RLS) policies are planned as an additional enforcement layer in a future hardening phase.

8.2 Tenant Isolation Rules

Every query that touches a tenant-owned entity must include `tenantId` in the filter condition. The API Gateway injects the authenticated `tenantId` into every downstream request header, and each service re-validates it before executing any database operation.

- Reads: every `SELECT` includes `WHERE tenantId = :tenantId`.
- Writes: every `INSERT` sets `tenantId` from the validated session context, never from user input.
- Cross-tenant operations are only possible via the Referral Service, which validates both tenants explicitly.

8.3 Tenant Provisioning Flow

1. Admin calls `POST /api/admin/tenants` with name and slug.
2. System creates Tenant record and a default admin User for that tenant.
3. System seeds a default set of Roles (Admin, Officer, Supervisor) for the tenant.
4. Admin receives credentials and can begin configuring workflows.

8.4 Cross-Tenant Data Considerations

Case data never crosses tenant boundaries except through an explicit Referral flow, which creates a copy of the case under the target tenant once the referral is accepted. Both the source and target cases maintain independent histories after that point.

9. Workflow Engine

9.1 Internal Transition Processing

When a transition request arrives at the Case Service, the engine follows this exact sequence:

5. Load the current case, including its workflowVersionId and currentStepId.
6. Load all transitions for the current step from WorkflowTransition where fromStepId = currentStepId.
7. Find the transition matching the requested actionLabel.
8. Validate that the requesting user holds the requiredRoleId for that transition.
9. Validate that the case status is not CLOSED.
10. Execute the database transaction (see Section 17 for Outbox Pattern details).
11. Return the updated case state to the caller.

9.2 Transition Validation Rules

Validation	Failure Response
Case exists and belongs to tenant	404 Not Found
Transition action exists for current step	422 Unprocessable Entity (invalid action)
User holds the required role	403 Forbidden
Case is not in CLOSED status	409 Conflict
WorkflowVersion is not archived	422 Unprocessable Entity

9.3 Transition Storage and Evaluation

Transition rules are stored as structured rows in the WorkflowTransition table (not as code or scripts). This means the engine is a simple data-driven lookup: given a step and an action label, there is at most one valid transition record. If no record matches, the action is rejected. This design keeps the engine stateless and predictable.

9.4 Database Transaction Scope

Every transition executes within a single database transaction that atomically: updates Case.currentStepId and Case.status, inserts a CaseHistory record, and inserts an OutboxEvent record. If any part fails, the entire transaction is rolled back. The Outbox record is the only way a Kafka event gets published; there is no direct Kafka call inside the transaction.

9.5 Conditional Branching

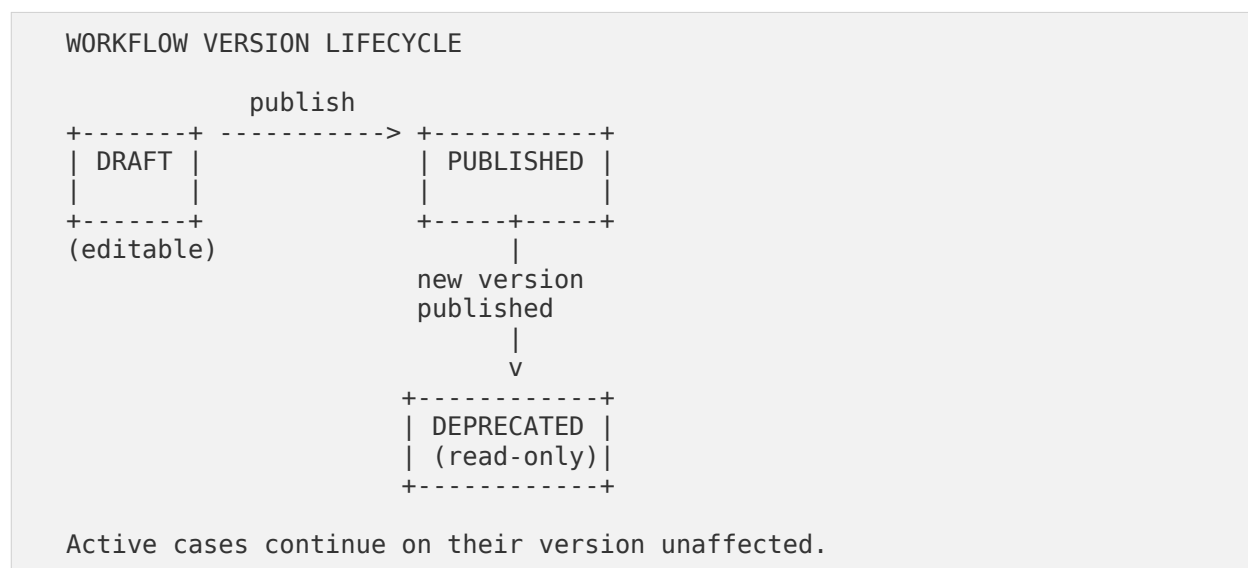
The current engine supports conditional branching through multiple transitions from the same step. For example, a Review step may have two transitions: Approve (to Approval) and Reject (to Applicant). The user chooses which action to take, and the engine enforces the role requirement on each branch independently. Time-based automatic transitions and parallel workflow execution are not supported in the current version and are documented as planned features in Section 32.

9.6 Case Creation

Creating a case always targets the most recently PUBLISHED WorkflowVersion for a given Workflow. The engine sets currentStepId to the step where isInitial = true. It inserts a CaseHistory record with fromStepId = null to mark the opening event.

10. Workflow Versioning Lifecycle

10.1 Version States



10.2 State Descriptions

Status	Behaviour
DRAFT	Editable. Steps and transitions can be added, changed, or removed. Cannot be used by new cases. Only one DRAFT per Workflow may exist at a time.
PUBLISHED	Immutable. New cases are created using this version. Only one PUBLISHED version may exist per Workflow at a time.
DEPRECATED	Read-only. Existing cases continue executing against this version and reach their final steps normally. No new cases can be created from it.

10.3 Publishing Process

Before a DRAFT version can be published, the system validates:

- Exactly one WorkflowStep has isInitial = true.
- At least one WorkflowStep has isFinal = true.

- Every non-final step has at least one outgoing transition.
- No transition references a step in a different WorkflowVersion.
- Every transition has a valid requiredRoleId belonging to the same tenant.

If validation passes, the previous PUBLISHED version is moved to DEPRECATED, the DRAFT is moved to PUBLISHED, and publishedAt and publishedBy are recorded.

10.4 Immutability After Publication

WorkflowVersion records in PUBLISHED or DEPRECATED status are fully immutable. No changes are allowed to their steps or transitions. This guarantees that all cases on a given version follow identical rules for their entire lifecycle, even if the workflow is redesigned many times after.

10.5 Case Migration

Manual migration of active cases from one workflow version to another is not supported in the current release. All active cases complete their lifecycle on the version they were created under. This is an intentional safety constraint that prevents mid-flight cases from being affected by workflow redesigns.

11. “Other Side Visibility” Tracking

11.1 What Users See

For every case, the Case Service computes and returns the following context alongside the case data:

- Current step name and description.
- The role(s) allowed to act at the current step.
- All available transition actions for the requesting user (filtered by their roles).
- The destination step and responsible role for each available action.
- Full chronological case history with actor, step changes, and comments.

11.2 Example API Response (GET /api/cases/:id)

```
{
  "id": "case-uuid",
  "title": "Investigation Request #1042",
  "status": "IN_PROGRESS",
  "currentStep": {
    "id": "step-uuid",
    "name": "Review",
    "allowedRoles": ["Officer"]
  },
  "availableActions": [
    {
      "actionLabel": "Approve",
```

```
    "toStep": "Approval",
    "responsibleRole": "Supervisor"
  },
  {
    "actionLabel": "Reject",
    "toStep": "Applicant",
    "responsibleRole": "Applicant"
  }
],
"history": [...]
}
```

12. Authentication and Access Control

12.1 Authentication Modes

The platform supports two authentication mechanisms:

- Session-based: the Auth Service issues an HTTP-only cookie containing a session ID, backed by Redis. Used for browser clients.
- JWT-based: the Auth Service issues a short-lived access token (RS256, 15-minute expiry) and a long-lived refresh token (7-day expiry, stored in Redis). Used for API and mobile clients.

12.2 Token Lifecycle

Token Type	Details
Access Token (JWT)	15-minute expiry. Signed with RS256. Contains userId, tenantId, and role list. Validated by the API Gateway on every request without a database call.
Refresh Token	7-day expiry. Opaque UUID stored in Redis keyed by userId. Exchanged for a new access token via POST /api/auth/refresh. Deleted from Redis on logout.
Session Cookie	HTTP-only, Secure, SameSite=Strict. Session metadata stored in Redis with a 24-hour TTL. Used as an alternative to JWT for browser flows.

12.3 Gateway Validation Flow

12. Request arrives at the API Gateway.
13. Gateway extracts the Bearer token (JWT) or session cookie.
14. For JWT: validates signature, checks expiry, decodes payload.
15. For session: validates session ID against Redis.
16. Gateway injects X-Tenant-ID, X-User-ID, X-User-Roles, and X-Correlation-ID headers into the forwarded request.
17. Downstream services trust these headers without re-validating the token.

12.4 Tenant Context Injection

The `tenantId` is always derived from the validated token, never from URL parameters or request body fields. This prevents any possibility of a user crafting a request that accesses another tenant's data by manipulating input.

13. Role-Based Access Control Model

13.1 Permission Model Structure

RBAC PERMISSION MODEL

```

User
+---(UserRole junction)---> Role(s)
                             |
                             (RolePermission junction)
                             |
                             Permission(s)

```

Step-Level Check:

```

WorkflowStep.allowedRoles
n user.currentRoles != empty => may ACT

```

Transition-Level Check:

```

Transition.requiredRoleId
in user.currentRoles          => may TRIGGER

```

Both conditions must pass independently.

13.2 Permission Layers

Layer	What It Controls
Global Permission (Role-Permission)	Platform-level capabilities such as <code>workflow:publish</code> , <code>case:create</code> , or <code>audit:read</code> . Checked at the API Gateway or individual service level.
Step-Level Permission (WorkflowStepRole)	Which roles may view and act on cases currently at a given workflow step. Enforced by the Case Service when listing cases and returning available actions.
Transition-Level Permission (WorkflowTransition.requiredRoleId)	Which single role may trigger a specific transition. Enforced by the workflow engine before executing the transition.

13.3 Runtime Evaluation

When a user requests to transition a case, the Case Service executes this evaluation in order:

18. Retrieve the case and verify tenantId matches the user's tenantId.
19. Load the WorkflowStep allowedRoles for the currentStepId.
20. Check: does the intersection of user roles and allowedRoles contain at least one entry?
If no: return 403.
21. Find the WorkflowTransition matching the requested actionLabel for this step.
22. Check: does the user hold the transition's requiredRoleId? If no: return 403.
23. Proceed to execute the transition.

13.4 Permission Model Properties

- Additive model: a user with multiple roles gains the union of all role capabilities. There are no deny rules.
- No inheritance: roles do not inherit from other roles in Version 1. Each role's permissions are explicit.
- No conflict resolution needed: since the model is purely additive, multiple roles on one user only expand access, never restrict it.
- Computed available actions: GET /api/cases/:id/actions returns the list of transitions the requesting user can actually trigger, computed at request time from their roles.

14. Security Design

14.1 Password Security

- Passwords are hashed with bcrypt at cost factor 12 before storage.
- Plain-text passwords are never logged, stored, or returned in any API response.
- Password complexity requirements are enforced at registration: minimum 10 characters, at least one uppercase letter, one number, and one special character.
- Failed login attempts are rate-limited: 10 attempts per 15 minutes per IP per tenant before temporary lockout.

14.2 JWT Security

- Access tokens are signed with RS256 (RSA 2048-bit) using a private key held only by the Auth Service.
- All other services verify tokens using the corresponding public key, distributed via a JWKS endpoint.
- Access token expiry is 15 minutes. Refresh tokens are 7 days.
- Compromised access tokens can be revoked by adding them to a Redis-backed denylist keyed by jti (JWT ID) until their natural expiry.
- Refresh tokens are rotated on every use: the old token is deleted and a new one is issued.

14.3 Transport Security

- HTTPS is enforced across all environments. HTTP requests are redirected to HTTPS at the load balancer or ingress.

- TLS 1.2+ is required; TLS 1.0 and 1.1 are disabled.
- HSTS (HTTP Strict Transport Security) is enabled with a minimum max-age of 1 year.

14.4 Input Security

- All request bodies are validated against JSON schemas using a validation middleware at each service entry point before business logic is reached.
- Prisma's parameterized queries protect against SQL injection throughout the platform.
- User-supplied strings are sanitized before storage to strip potentially dangerous HTML and script tags.
- Content Security Policy (CSP) headers are set on all responses from the API Gateway.

14.5 Session Security

- Session cookies are set with HttpOnly, Secure, and SameSite=Strict flags.
- CSRF tokens are required for session-based requests that mutate state.
- Sessions are invalidated immediately on logout by deleting the Redis session record.

14.6 Rate Limiting

Endpoint Group	Limit
POST /api/auth/login	10 requests per 15 minutes per IP
POST /api/auth/register	5 requests per hour per IP
POST /api/auth/refresh	30 requests per hour per user
All other authenticated endpoints	300 requests per minute per user
All endpoints (global)	1000 requests per minute per IP

14.7 Secret Management

- All secrets (database credentials, JWT private keys, Redis passwords) are injected via environment variables.
- In production, secrets are managed by a secrets manager (AWS Secrets Manager or HashiCorp Vault) and never committed to source control.
- Secret rotation is performed without downtime using zero-downtime key rotation patterns (supporting both old and new keys during transition).

14.8 Tenant Isolation as a Security Control

Tenant isolation (Section 8) is treated as a security control, not just a data organization concern. Any bug that allows cross-tenant data access is classified as a critical security vulnerability and prioritized accordingly.

15. API Specifications

All endpoints are accessed through the API Gateway on port 3000. The Gateway handles authentication, tenant context, and routing. Auth values: “None” = public endpoint; “JWT” = valid access token required; “JWT + Admin Role” = requires Admin role within the tenant.

15.1 Auth Service (/api/auth)

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register a new user. Body: { tenantSlug, email, password, firstName, lastName }
POST	/api/auth/login	None	Login. Returns { accessToken, refreshToken, user }
POST	/api/auth/refresh	Refresh Token	Exchange a refresh token for a new access token
POST	/api/auth/logout	JWT	Invalidate the current session and refresh token
GET	/api/auth/me	JWT	Return the authenticated user's profile
PUT	/api/auth/me/password	JWT	Change the authenticated user's password

15.2 RBAC Service (/api/roles, /api/users)

Method	Endpoint	Auth	Description
GET	/api/roles	JWT	List all roles for the current tenant
POST	/api/roles	JWT + Admin	Create a new role. Body: { name, description }
GET	/api/roles/:id	JWT	Get a role and its associated permissions
PUT	/api/roles/:id	JWT + Admin	Update role name or description
DELETE	/api/roles/:id	JWT + Admin	Delete role (rejected if users are assigned)
POST	/api/roles/:id/permissions	JWT + Admin	Add a permission to a role
DELETE	/api/roles/:id/permissions/:permId	JWT + Admin	Remove a permission from a role
GET	/api/users/:userId/roles	JWT	List roles assigned to a user

Method	Endpoint	Auth	Description
POST	/api/users/:userId/roles	JWT + Admin	Assign a role to a user. Body: { roleId }
DELETE	/api/users/:userId/roles/:roleId	JWT + Admin	Remove a role from a user

15.3 Workflow Service (/api/workflows)

Method	Endpoint	Auth	Description
POST	/api/workflows	JWT	Create a new workflow (begins in DRAFT). Body: { name, description }
GET	/api/workflows	JWT	List all workflows for the tenant
GET	/api/workflows/:id	JWT	Get workflow with its latest version details
POST	/api/workflows/:id/steps	JWT	Add a step to the current DRAFT. Body: { name, description, isInitial, isFinal, sortOrder }
PUT	/api/workflows/:id/steps/:stepId	JWT	Update a draft step
DELETE	/api/workflows/:id/steps/:stepId	JWT	Remove a step from the current DRAFT
POST	/api/workflows/:id/steps/:stepId/roles	JWT	Assign a role to a workflow step
POST	/api/workflows/:id/transitions	JWT	Add a transition to the current DRAFT
DELETE	/api/workflows/:id/transitions/:transId	JWT	Remove a transition from the DRAFT
POST	/api/workflows/:id/publish	JWT + Admin	Validate and publish the DRAFT (creates new version)
GET	/api/workflows/:id/versions	JWT	List all versions of a workflow with their statuses
GET	/api/workflows/:id/versions/:version	JWT	Get a specific workflow version with all steps and transitions

15.4 Case Service (/api/cases)

Method	Endpoint	Auth	Description
POST	/api/cases	JWT	Create a new case. Body: { workflowId, title, description, caseData }
GET	/api/cases	JWT	List cases visible to the user (filtered by role and step). Supports ?status=, ?stepId=, ?assignedTo=, ?page=, ?limit=
GET	/api/cases/:id	JWT	Get a case with currentStep, availableActions, and recent history
GET	/api/cases/:id/actions	JWT	Get the transitions the current user can trigger on this case
POST	/api/cases/:id/transition	JWT	Execute a transition. Body: { actionLabel, comment }
GET	/api/cases/:id/history	JWT	Get the full chronological history of a case
PUT	/api/cases/:id/assign	JWT	Assign the case to a user. Body: { userId }
PUT	/api/cases/:id/data	JWT	Update dynamic caseData fields. Body: { caseData }

Case Transition Request / Response Example

```
// POST /api/cases/case-uuid/transition
// Request body:
{
  "actionLabel": "Approve",
  "comment": "All supporting documents verified."
}

// Response 200:
{
  "id": "case-uuid",
  "status": "IN_PROGRESS",
  "previousStep": { "id": "...", "name": "Review" },
  "currentStep": { "id": "...", "name": "Approval" },
  "availableActions": [
    { "actionLabel": "Approve", "toStep": "Closed", "responsibleRole":
"Director" }
  ],
  "updatedAt": "2024-11-15T09:30:00.000Z"
}
```

15.5 Referral Service (/api/referrals)

Method	Endpoint	Auth	Description
POST	/api/referrals	JWT	Create a referral. Body: { targetTenantSlug, sourceCaseId, note }
GET	/api/referrals	JWT	List referrals (outbound and inbound). Supports ?status=, ?direction=
GET	/api/referrals/:id	JWT	Get referral details
PUT	/api/referrals/:id/accept	JWT	Accept a referral and create the case in the target tenant. Body: { workflowId }
PUT	/api/referrals/:id/reject	JWT	Reject a referral. Body: { reason }

15.6 Audit Service (/api/audit)

Method	Endpoint	Auth	Description
GET	/api/audit/events	JWT + Admin	Query audit events. Supports ?tenantId=, ?eventType=, ?entityType=, ?from=, ?to=, ?page=, ?limit=
GET	/api/audit/events/case/:caseId	JWT	All audit events for a specific case in chronological order
GET	/api/audit/events/user/:userId	JWT + Admin	All audit events associated with a specific user

15.7 Notification Service (/api/notifications)

Method	Endpoint	Auth	Description
GET	/api/notifications	JWT	List notifications for the current user. Supports ?unreadOnly=true
PUT	/api/notifications/:id/read	JWT	Mark a notification as read
PUT	/api/notifications/read-all	JWT	Mark all notifications as read
WS	/ws/notifications	JWT (header)	WebSocket connection for real-time notification push

15.8 Common HTTP Response Codes

Code	Meaning in IACMS
200 OK	Successful read or update
201 Created	Resource created successfully
400 Bad Request	Request body failed schema validation
401 Unauthorized	Missing or invalid authentication token
403 Forbidden	Token valid but user lacks the required role
404 Not Found	Resource does not exist or belongs to another tenant
409 Conflict	Operation rejected due to current resource state (e.g. case is CLOSED)
422 Unprocessable Entity	Business rule violation (e.g. invalid transition action)
429 Too Many Requests	Rate limit exceeded
500 Internal Server Error	Unhandled server-side error
503 Service Unavailable	Downstream dependency unavailable; retry with backoff

16. Event-Driven Architecture

16.1 Kafka Topics

Topic	Partitioned By	Retention	Purpose
iacms.case.events	tenantId	7 days	All case lifecycle events (created, transitioned, assigned, closed)
iacms.workflow.events	tenantId	7 days	Workflow version lifecycle events (published, deprecated)
iacms.referral.events	tenantId	7 days	Referral lifecycle events (created, accepted, rejected)

Topic	Partitioned By	Retention	Purpose
iacms.audit.ingestion	tenantId	30 days	All audit-worthy events from every service, consumed by Audit Service
iacms.notification.dispatch	userId	1 day	Notification triggers consumed by the Notification Service

16.2 Producer and Consumer Map

Topic	Producers	Consumers
iacms.case.events	Case Service	Audit Service, Notification Service
iacms.workflow.events	Workflow Service	Audit Service
iacms.referral.events	Referral Service	Audit Service, Notification Service, Case Service
iacms.audit.ingestion	All Services	Audit Service
iacms.notification.dispatch	Case Service, Referral Service	Notification Service

16.3 Event Types

Event Type	Produced By	Description
CASE_CREATED	Case Service	A new case was created from a published workflow
CASE_TRANSITIONED	Case Service	A case moved from one workflow step to another
CASE_ASSIGNED	Case Service	A case was assigned to a specific user
CASE_CLOSED	Case Service	A case reached a final workflow step
WORKFLOW_PUBLISHED	Workflow Service	A new workflow version was published
WORKFLOW_DEPRECATED	Workflow Service	A workflow version was deprecated by a newer one
REFERRAL_CREATED	Referral Service	A cross-agency referral was initiated
REFERRAL_ACCEPTED	Referral Service	A referral was accepted and a new case was created
REFERRAL_REJECTED	Referral Service	A referral was rejected by the target agency
USER_REGISTERED	Auth Service	A new user registered on the platform

Event Type	Produced By	Description
USER_LOGIN	Auth Service	A user successfully authenticated
ROLE_ASSIGNED	RBAC Service	A role was assigned to or removed from a user

16.4 Standard Event Envelope

Every event published to Kafka follows this common envelope structure:

```
{
  "eventId": "uuid-v4",
  "eventType": "CASE_TRANSITIONED",
  "schemaVersion": "1.0",
  "tenantId": "uuid-v4",
  "timestamp": "2024-11-15T09:30:00.000Z",
  "source": "case-service",
  "correlationId": "uuid-v4",
  "payload": {
    "caseId": "uuid-v4",
    "caseTitle": "Investigation Request #1042",
    "fromStepId": "uuid-v4",
    "fromStepName": "Review",
    "toStepId": "uuid-v4",
    "toStepName": "Approval",
    "transitionId": "uuid-v4",
    "actionLabel": "Approve",
    "performedBy": "uuid-v4",
    "comment": "All documents verified"
  }
}
```

16.5 Reliability and Error Handling

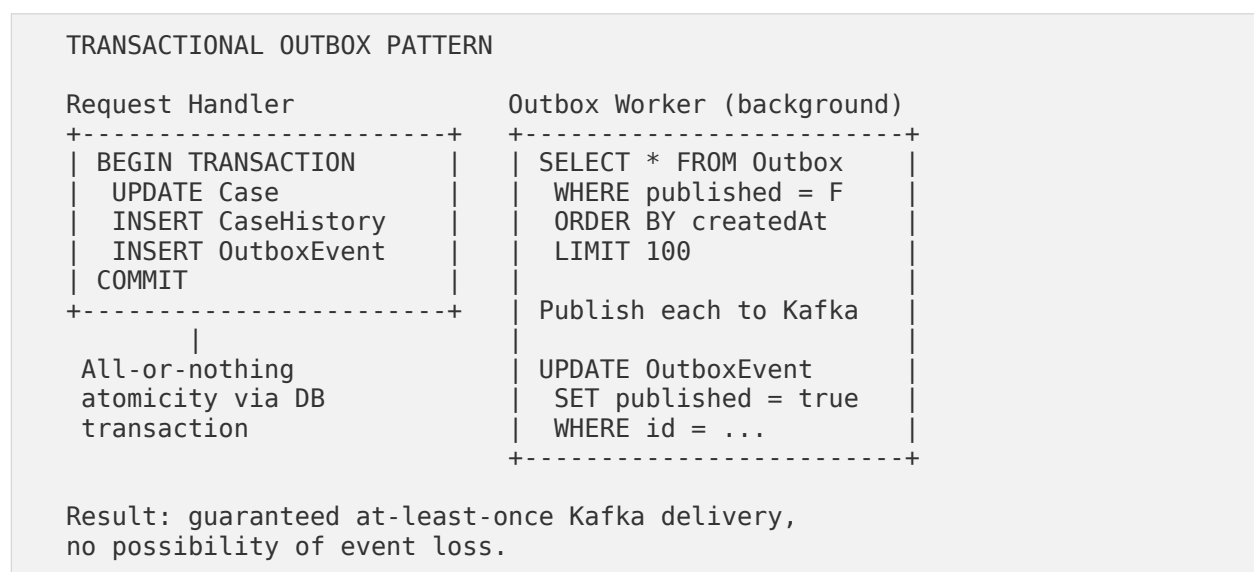
- Consumer groups: each consuming service has its own consumer group, so multiple services consuming the same topic each receive every message independently.
- Idempotency: consumers are designed to be idempotent by checking whether an eventId has already been processed before acting. Duplicate delivery (at-least-once semantics) is safe.
- Dead Letter Queue: if a consumer fails to process a message after 3 retries, the message is moved to a DLQ topic (e.g. iacms.case.events.dlq) for manual inspection and reprocessing.
- Ordering: events within the same tenantId are delivered in order within a single partition. Cross-partition ordering is not guaranteed.
- Schema registry: event schemas are registered and versioned so consumers can safely evolve alongside producers.

17. Transaction Management

17.1 The Distributed Transaction Problem

Case transitions require three coordinated writes: updating the Case record, inserting a CaseHistory record, and publishing a Kafka event. If Kafka is called directly inside a database transaction and then fails, the database commit may succeed while the event is lost. If Kafka is called after the commit and then the service crashes, the event is also lost. Neither approach is safe in a distributed system.

17.2 The Transactional Outbox Pattern



17.3 Outbox Worker

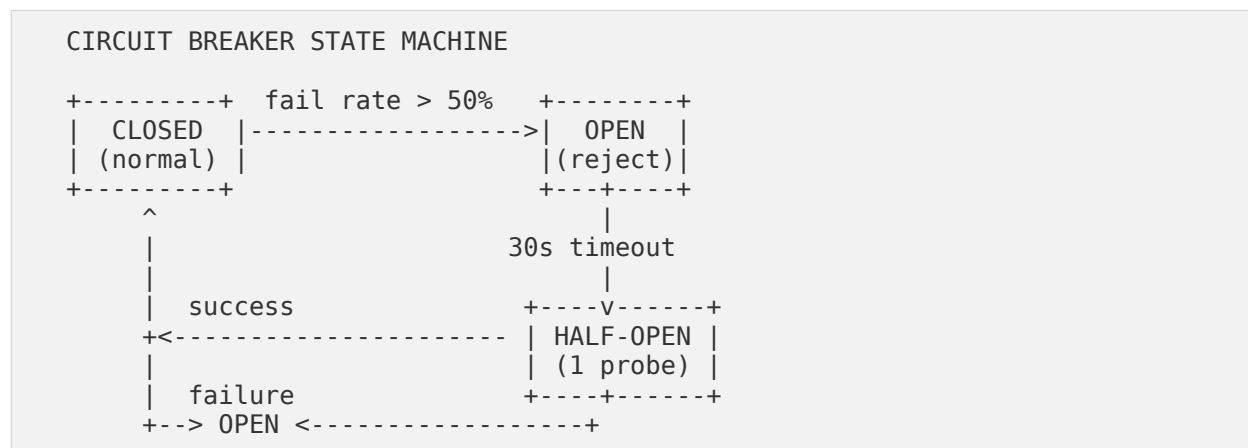
The Outbox Worker is a background process (running within the Case Service) that polls the OutboxEvent table every 500ms for rows where published = false. It publishes each event to the appropriate Kafka topic and then marks it as published. If the Kafka publish call fails, the row remains unpublished and will be retried on the next poll cycle.

17.4 Saga Pattern for Multi-Service Flows

Operations that span multiple services (such as accepting a referral, which requires creating a case in the target tenant) are implemented as choreography-based sagas over Kafka. Each service listens for a trigger event, performs its local operation, and publishes a result event. If a step fails, a compensating event is published to undo previous steps. This avoids distributed locks and two-phase commits.

18. Failure Handling and Resilience

18.1 Circuit Breaker Pattern



Circuit breakers (implemented using the opossum library) wrap all inter-service HTTP calls and external dependency connections. The breaker opens after a 50% error rate over a 10-second window and automatically probes after a 30-second reset timeout.

18.2 Per-Dependency Failure Scenarios

Dependency	Failure Behaviour	Recovery
Kafka unavailable	OutboxEvents accumulate in PostgreSQL. New requests continue succeeding. No case transitions are lost.	Events are replayed automatically when Kafka recovers. No manual intervention needed.
Redis unavailable	Session validation falls through to database-backed JWT verification. Performance degrades slightly. Cache misses increase DB load.	Redis reconnects automatically. Cache is repopulated on subsequent reads.
Database timeout	Transaction rolls back. 503 returned to client. Case state unchanged. No partial update is possible.	Client retries with exponential backoff. DBA investigates connection pool or query bottleneck.
Upstream service unreachable	API Gateway circuit breaker opens after threshold. Returns 503 with Retry-After header.	Circuit probes the upstream service. Gateway resumes routing when service recovers.
Notification Service down	Notifications are queued in Kafka. Case transitions complete successfully without waiting.	Notification Service replays its consumer group offset on restart.

18.3 Retry Policy

- HTTP calls between services: exponential backoff with jitter, 3 attempts, initial delay 100ms, max delay 2s.
- Kafka consumer failures: automatic retry with delay up to 3 times before routing to DLQ.
- Database connection failures: Prisma connection pool handles reconnection automatically.

18.4 Health Check Endpoints

Every service exposes two health endpoints:

- GET /health — liveness probe. Returns 200 if the process is running. Does not check dependencies.
- GET /health/ready — readiness probe. Returns 200 only if the service can successfully connect to PostgreSQL, Redis, and Kafka. Returns 503 if any dependency is unreachable.

Kubernetes uses liveness probes to restart crashed pods and readiness probes to stop routing traffic to pods that are still initializing or have lost dependency connections.

18.5 Graceful Shutdown

On receiving SIGTERM, each service: stops accepting new requests, waits for in-flight requests to complete (up to 30 seconds), flushes pending log writes, closes database connections gracefully, and then exits with code 0. This prevents data corruption during rolling deployments.

19. Performance Considerations

19.1 Database

Area	Strategy
Connection pooling	Prisma connection pool sized at 10 connections per service instance by default. Adjust based on PostgreSQL max_connections and number of service replicas.
Tenant-scoped queries	All list queries include tenantId in the WHERE clause, using the (tenantId) index to avoid full table scans.
Case listing	The (tenantId, status) compound index on Case enables efficient dashboard queries without filtering the entire case table.
History retrieval	CaseHistory is append-only and indexed by (caseId, createdAt). History queries are efficient regardless of case volume.
JSONB caseData	Avoid indexing JSONB fields unless specific keys are queried frequently. If required, use GIN indexes on specific JSONB keys.
Audit queries	AuditEvent is indexed by (tenantId, createdAt) for time-range compliance queries. For high-volume audit traffic, consider partitioning AuditEvent by month.

19.2 Caching Strategy

Cache Target	Strategy
Workflow definitions	Published WorkflowVersions with their steps and transitions are cached in Redis with a 1-hour TTL. Cache is invalidated when a new version is published.
User roles	The user's role list is embedded in the JWT access token (15-minute TTL). No Redis lookup needed for role checks.
Session data	Active sessions are stored in Redis with a 24-hour TTL. Session lookup is an O(1) Redis GET.
Frequently listed cases	Case list results can be cached per (tenantId, status, page) with a 30-second TTL for read-heavy dashboards.

19.3 Pagination

All list endpoints that may return large result sets support cursor-based pagination using the createdAt timestamp as the cursor. This avoids the performance degradation of OFFSET-based pagination on large datasets. Clients receive a nextCursor in the response and pass it as a query parameter in the next request.

19.4 Expected Workload Characteristics

Metric	Guidance
Cases per tenant	System tested up to 500,000 active cases per tenant. Archiving closed cases older than 2 years is recommended at scale.
Transitions per case	No hard limit. CaseHistory is append-only. Average expected: 5-20 transitions per case.
Workflow definitions	No hard limit. Typically 5-50 workflows per tenant.
Concurrent users	Horizontal scaling of stateless services supports linear concurrency growth.

20. Observability and Monitoring

20.1 Structured Logging

Every service logs using Winston in JSON format. Each log entry includes:

```
{
  "timestamp": "2024-11-15T09:30:00.123Z",
  "level": "info",
```

```
"service": "case-service",
"correlationId": "uuid-v4",
"tenantId": "uuid-v4",
"userId": "uuid-v4",
"message": "Case transition executed",
"meta": {
  "caseId": "uuid-v4",
  "action": "Approve",
  "durationMs": 42
}
}
```

20.2 Correlation IDs

The API Gateway generates a UUID correlation ID for every incoming request and injects it as the X-Correlation-ID header into all downstream service calls and Kafka messages. This ID is included in every log entry, making it possible to trace the complete execution path of any request across all services using a single log search.

20.3 Metrics (Prometheus)

Each service exposes a /metrics endpoint in Prometheus format. Key metrics collected:

Metric	Description
http_request_duration_seconds	Histogram of HTTP request latency per route and status code
http_requests_total	Counter of requests per route, method, and status code
case_transitions_total	Counter of case transitions by tenant and action
kafka_consumer_lag	Current consumer group lag per topic and partition
db_connection_pool_size	Active and idle database connections per service
redis_hit_rate	Cache hit vs miss ratio for Redis lookups
outbox_pending_events	Number of unpublished OutboxEvent rows (alerts if this grows)

20.4 Distributed Tracing

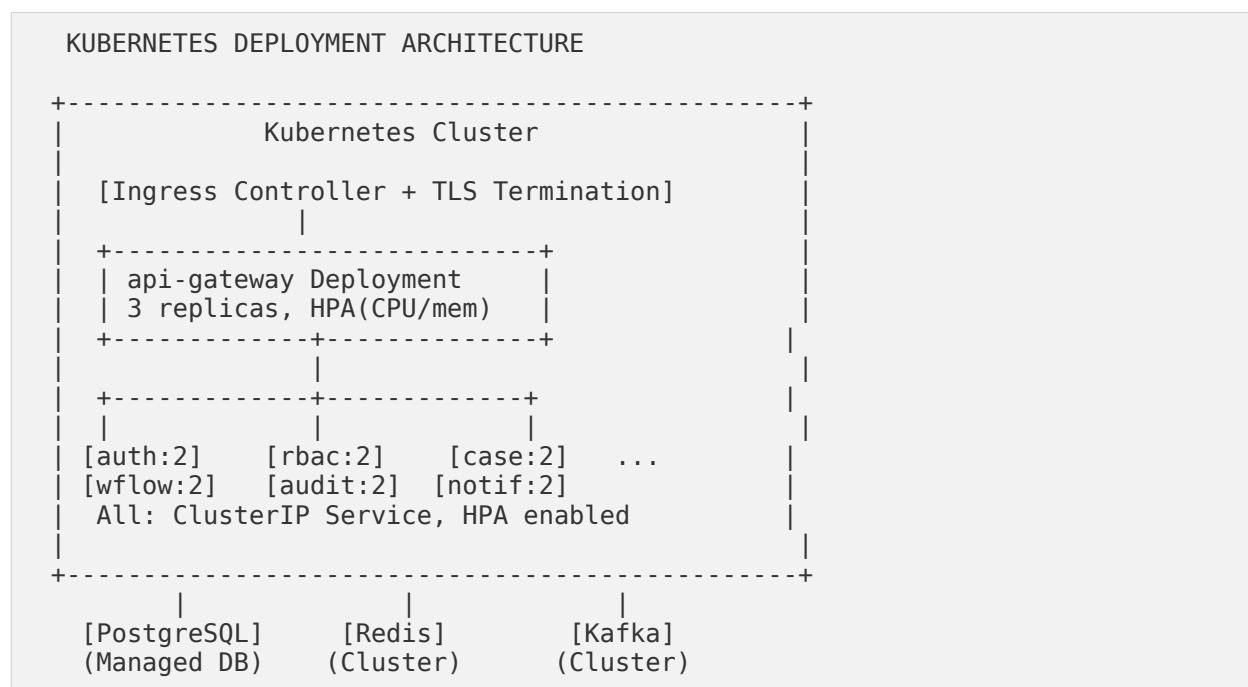
OpenTelemetry instrumentation is integrated into the API Gateway and all backend services. Traces are exported to Jaeger (or a compatible backend) and can be correlated with logs via the shared correlationId. Each service-to-service call creates a child span within the originating request's trace.

20.5 Alerting Thresholds

Alert Condition	Severity
HTTP 500 error rate > 1% over 5 minutes	Critical
P95 request latency > 2 seconds for any endpoint	Warning
Kafka consumer lag > 10,000 messages on any topic	Warning
OutboxEvent pending count > 500 for more than 5 minutes	Critical
Database connection pool exhausted on any service	Critical
Health check readiness failure on any pod	Warning (auto-handled by K8s)
JWT public key JWKS endpoint unreachable	Critical

21. Deployment Architecture

21.1 Kubernetes Architecture



21.2 Kubernetes Resource Summary

Component	K8s Kind	Replicas	Notes
API Gateway	Deployment	3 (min)	LoadBalancer or Ingress with TLS. HPA scales to 10.
Backend Services (x8)	Deployment	2 (min)	ClusterIP. HPA scales to 5 each based on CPU.
PostgreSQL	StatefulSet or Managed	1 primary + 1 replica	AWS RDS / GCP Cloud SQL recommended for production.
Redis	StatefulSet or Managed	3 (cluster mode)	Sentinel mode acceptable for smaller deployments.
Kafka	StatefulSet or Managed	3 brokers	Confluent Cloud or MSK removes operational burden.
Outbox Workers	Deployment	1 per case/referral svc	Only one instance may run per service to avoid duplicate publishes.

21.3 Environment Strategy

Environment	Purpose
Development	Local Docker Compose. Developers run services individually. No resource limits.
Staging	Full Kubernetes cluster mirroring production topology. Used for integration testing and pre-release validation.
Production	Multi-AZ Kubernetes deployment. Managed database and message broker services. Full observability stack active.

21.4 CI/CD Pipeline

Stage	Actions
Lint	ESLint and Prettier checks on all changed files.
Unit Tests	Jest unit tests per service. Minimum 80% coverage gate.
Integration Tests	Docker Compose spins up dependencies; service integration tests run against real DB, Redis, Kafka.
Build	Docker image built per changed service. Tagged with git SHA.
Push	Image pushed to container registry (ECR / GCR).
Deploy to Staging	Helm chart updated with new image tag. Staged rollout with readiness gate.

Stage	Actions
Smoke Tests	Automated API smoke tests run against staging to confirm critical paths work.
Deploy to Production	Manual approval gate. Helm rolling update. Automatic rollback if readiness probes fail.

22. Scalability Strategy

22.1 Horizontal Service Scaling

All IACMS services are designed to be stateless: user session state lives in Redis, application state lives in PostgreSQL, and event queues live in Kafka. This means any service can be scaled horizontally by adding replicas without coordination. Kubernetes HPA scales each service independently based on CPU utilization (target: 70%) and memory pressure.

22.2 Database Scaling

Strategy	Details
Read replicas	PostgreSQL read replicas serve all read-only queries (case listing, history, audit queries). Prisma datasource is configured to route reads to the replica and writes to the primary.
Connection pooling	PgBouncer in transaction pooling mode sits between services and PostgreSQL, multiplexing many short-lived Prisma connections into fewer long-lived DB connections.
Table partitioning	AuditEvent and CaseHistory tables can be range-partitioned by month once volume exceeds 50 million rows, without changing the application schema.
Archiving	Closed cases older than a configurable threshold can be moved to a cold-storage schema or read-replica-only database to keep the primary database size manageable.

22.3 Kafka Scaling

- Each topic is created with a number of partitions equal to the expected peak parallelism (typically 10-20 partitions per topic in production).
- Topics partitioned by tenantId ensure that all events for a single tenant are processed in order by a single consumer instance, preventing race conditions.
- Increasing the number of partitions and consumer group instances scales throughput linearly.

22.4 Redis Scaling

- Redis Cluster mode distributes keys across multiple nodes, scaling both memory capacity and read throughput.

- Cache TTLs are tuned to balance freshness against cache efficiency. Workflow definitions (1 hour TTL) are the highest-value cache target.

22.5 Long-Term Tenant Growth

The row-level multi-tenancy model scales well until very large tenants create lock contention or index bloat. At that scale, high-volume tenants can be migrated to dedicated schemas or database instances, with the shared application layer updated to route their requests to the tenant-specific connection. This migration is transparent to API clients.

23. Architecture Decision Records

The following Architecture Decision Records (ADRs) document the rationale behind major technology and design choices. They are intended to help future engineers understand why the current architecture exists and what trade-offs were accepted.

ADR-001: Microservices Architecture

Field	Detail
Status	Accepted
Decision	IACMS is structured as a collection of independently deployable microservices rather than a monolith or modular monolith.
Rationale	Independent deployability allows each service to be scaled, updated, and restarted without affecting the others. Fault isolation means a failure in the Notification Service does not bring down Case transitions. Team autonomy is supported by clear service boundaries.
Trade-offs	Increased operational complexity (multiple deployment targets, distributed tracing required). Network latency between services. More complex local development setup. Accepted because the system is expected to grow to multiple teams and require independent scaling of different services.
Alternatives Considered	Modular Monolith: simpler operations but would require more discipline to maintain boundary integrity. Recommended as a starting point if rebuilding from scratch with a smaller team.

ADR-002: Node.js as Application Runtime

Field	Detail
Status	Accepted
Decision	All microservices are written in Node.js with Express.
Rationale	Excellent async I/O performance for the HTTP-heavy request patterns of a case management API. Consistent language across services reduces context switching. Rich NPM ecosystem provides well-maintained libraries for JWT,

Field	Detail
	Kafka, Redis, and WebSockets.
Trade-offs	Single-threaded event loop is not ideal for CPU-intensive workloads. However, IACMS has no CPU-intensive service; all work is I/O-bound (database reads, Kafka publishes).
Alternatives Considered	Go: better performance and lower memory footprint, but steeper learning curve and less mature ORMs. Python: excellent for ML-adjacent work but less suited for high-concurrency HTTP services.

ADR-003: PostgreSQL as Primary Database

Field	Detail
Status	Accepted
Decision	PostgreSQL is the sole primary data store for all transactional data.
Rationale	ACID compliance is essential for workflow transitions, where partial writes must never succeed. JSONB support handles the dynamic caseData field without requiring a document database. Mature ecosystem with excellent tooling for migrations (Prisma), monitoring, and backup.
Trade-offs	Vertical scaling has limits; horizontal write scaling requires sharding which adds complexity. Accepted because the expected write throughput for a government case management system is moderate.
Alternatives Considered	MongoDB: better for schema-less documents but weaker ACID guarantees make workflow state transitions risky. MySQL: similar to PostgreSQL but with less advanced JSONB support and weaker window function performance.

ADR-004: Apache Kafka as Event Bus

Field	Detail
Status	Accepted
Decision	Kafka is the asynchronous event bus for all inter-service communication that does not require a synchronous response.
Rationale	Durable message storage (disk persistence) means events survive consumer downtime. Consumer groups allow multiple independent services to each receive a copy of every event. Replay capability allows the Audit Service to be rebuilt from the event log. Ordered delivery within partitions supports the Outbox Pattern reliably.
Trade-offs	Operationally more complex than a simple message queue. Kafka is overkill for low-volume deployments. Accepted because the platform is expected to handle high audit event volumes and requires reliable event replay capability.

Field	Detail
Alternatives Considered	RabbitMQ: simpler to operate but messages are not durably persisted by default and replay is not natively supported. Redis Pub/Sub: fast but not durable; messages are lost if no consumer is connected.

ADR-005: Redis for Session and Cache

Field	Detail
Status	Accepted
Decision	Redis is used for session storage and application caching.
Rationale	Sub-millisecond read/write performance for session lookups. Native TTL support makes session expiry automatic. Supports both session-based and JWT-based flows through the same infrastructure.
Trade-offs	In-memory data is lost if Redis is restarted without persistence configured. Mitigated by enabling Redis AOF persistence and deploying in cluster mode with replication.
Alternatives Considered	Database-backed sessions: correct but too slow for the frequency of session validation at the API Gateway. Memcached: no native persistence, no TTL per key, less feature-rich.

ADR-006: Prisma as ORM

Field	Detail
Status	Accepted
Decision	Prisma is used as the ORM and database schema management tool across all services via a shared schema.
Rationale	Type-safe query builder eliminates entire classes of runtime type errors. Schema-first migrations are version-controlled and auditable. The shared schema in the monorepo enforces a single source of truth for the database model.
Trade-offs	Prisma's query builder does not support all advanced PostgreSQL features natively; complex queries may require raw SQL. Prisma Client generation adds a build step.
Alternatives Considered	TypeORM: mature and feature-rich but more verbose and less type-safe in practice. Raw SQL (pg): maximum control but loses type safety and requires manual migration management.

24. Audit and Logging

24.1 Why Audit Logging Matters

- Accountability: every action is traceable to a specific user.
- Compliance: government deployments often require a full, immutable record of case actions.
- Debugging: the audit trail can reconstruct the exact sequence of events that led to a system state.

24.2 What Is Logged

- Who performed the action (performedBy user ID).
- When the action occurred (createdAt timestamp, millisecond precision).
- Which entity was affected (entityType, entityId).
- What changed (eventType, plus a metadata JSONB with before/after values where relevant).
- The correlation ID linking the audit event to the originating HTTP request.

24.3 Audit Architecture

Services publish audit-relevant events to the `iacms.audit.ingestion` Kafka topic. The Audit Service consumes these events and writes them to the `AuditEvent` table. This decoupling ensures that audit writes never block the primary request path and that the Audit Service can be rebuilt from the Kafka log if needed.

25. Notifications

25.1 Notification Types

- In-app (real-time): delivered via a WebSocket connection to the Notification Service. Clients receive notifications immediately when a case they are involved in transitions.
- Email: sent via a transactional email provider (e.g. SendGrid). Used for high-priority events such as new referrals and case assignments.

The current notification implementation (WebSocket + email) is maintained as-is. Extensions to SMS or push notification channels are planned for a future release.

25.2 Trigger Events

- A case the user is assigned to transitions to a new step.
- A case is assigned to the user.
- A referral is sent to the user's tenant.
- A referral the user created is accepted or rejected.

26. Local Development Setup

26.1 Prerequisites

- Docker Desktop or Docker Engine (with Docker Compose V2)
- Node.js 20 or higher
- npm 10 or higher
- Postman (recommended for API testing)

26.2 Start Infrastructure Services

```
cd infrastructure
docker-compose up -d postgres redis kafka zookeeper
```

26.3 Apply Database Migrations and Seed Data

```
cd ..
npx prisma migrate deploy
node prisma/seed.js
```

26.4 Start Individual Services

```
cd services/auth-service && npm start
# Repeat for other services as needed
```

26.5 Useful Root Commands

```
npm run docker:up      # Start all infrastructure
npm run docker:down    # Stop all infrastructure
npm run prisma:migrate # Run pending migrations
npm run prisma:studio  # Open Prisma Studio (DB browser)
npm run db:seed        # Seed reference data
npm run test:gateway   # Run Gateway test suite
```

27. API Conventions

All APIs are exposed through the API Gateway on port 3000 and routed to backend services based on path patterns.

- All request and response bodies use JSON with Content-Type: application/json.
- Dates and timestamps are ISO 8601 strings in UTC (e.g. 2024-11-15T09:30:00.000Z).
- IDs are UUID v4 strings.
- List endpoints return { data: [...], total: N, nextCursor: '...' }.
- Error responses return { error: 'CODE', message: '...' }.

- The X-Correlation-ID header is included in every response and should be referenced in bug reports.

28. Testing and Validation

The repository includes service-specific test folders. Validation approaches:

- Unit tests (Jest): pure function logic, workflow engine validation rules, permission checks.
- Integration tests (Jest + testcontainers): each service tested against a real PostgreSQL and Redis instance spun up in Docker.
- API tests (Postman collections): full request/response validation for all major endpoints including edge cases.
- Migration validation: all Prisma migrations are applied to a clean database and seeded before any test run.
- Load tests: k6 scripts for the most critical paths (case creation, case transition) are run before major releases.

29. Documentation Map

Document	Contents
README.md	Project overview and quick-start guide
docs/ SYSTEM_OVERVIEW.md	Architectural narrative
docs/ DOCUMENTATION.md	Technical reference material
docs/ KAFKA_INTEGRATION.md	Kafka topic and consumer configuration
docs/ TENANT_ISOLATION.md	Multi-tenancy design notes and enforcement rules
docs/ SESSION_AUTHENTICATI ON.md	Session authentication flow
PHASES.md	Roadmap and release phase planning
This document	Complete consolidated design and engineering reference

30. Current Project Status

The platform is a working multi-service system with workflow definition, case lifecycle management, authentication, RBAC, referrals, audit, and notification capabilities implemented. The architecture is structured for extension and production hardening.

Known gaps for production readiness: observability stack not yet deployed (Prometheus, Jaeger), CI/CD pipeline not yet configured, automated integration test coverage is partial, and production Kubernetes manifests are in draft.

31. Risks and Challenges

- Over-engineering workflows: tenant workflow editors should guide users toward simple, valid configurations. Complex branching should be gated behind documentation.
- Weak validation: if transition and role validation rules are inconsistently applied, data integrity issues accumulate silently. The validation sequence in Section 9 must be followed without exception.
- Tenant isolation bugs: any cross-tenant data leak is a critical security incident. All changes to query logic must be reviewed with tenant filtering in mind.
- Outbox Worker reliability: if the Outbox Worker crashes repeatedly, the pending event backlog grows and Kafka consumers fall behind. Monitoring the `outbox_pending_events` metric is essential.
- Schema migration coordination: in a multi-service monorepo sharing one Prisma schema, all services must be deployed in a backward-compatible manner when schema changes are made.

32. Notes for Future Development

- Complete end-to-end validation across all services.
- Expand automated test coverage, particularly for the workflow engine and RBAC enforcement.
- Deploy the full observability stack (Prometheus, Grafana, Jaeger, ELK).
- Implement the CI/CD pipeline and Kubernetes manifests for staging and production.
- Add support for conditional workflow branching based on case data values.
- Add time-based automatic transitions (e.g. escalate after 48 hours).
- Add support for parallel workflow stages.
- Implement PostgreSQL Row Level Security as an additional tenant isolation enforcement layer.
- Add a tenant-facing workflow designer UI.
- Support tenant case migration between workflow versions (admin-controlled).

33. Conclusion

IACMS is a workflow-driven multi-tenant case management platform built on a microservices architecture with structured event-driven communication. Its success depends on three things holding together over time: a correct and predictable workflow execution engine with proper transaction management, strict tenant data isolation enforced at every layer, and comprehensive observability that makes production issues diagnosable and recoverable.

This document addresses the full engineering scope of the system: component architecture, database schema, API contracts, event schemas, security model, resilience patterns, deployment topology, and the rationale behind every major design decision. Future engineers should treat the Architecture Decision Records in Section 23 as the authoritative explanation of why the system is built the way it is, and update them whenever a significant architectural change is made.